- They do not catch errors inside event handlers.
- They do not catch errors in asynchronous code (e.g., setTimeout or requestAnimationFrame callbacks).
- They do not catch errors during server-side rendering.
- They do not catch errors thrown in the error boundary component itself (rather than its children).

## A.6.2 Global error hooks

React 19 introduces two new optional hooks for centralized error processing at the root level, in addition to `onRecoverableError`. Now there are three hooks:

- `onCaughtError`—Invoked when an error is caught by an Error Boundary.
- `onUncaughtError`—Invoked for uncaught errors not handled by an Error Boundary.
- `onRecoverableError`—Invoked for recoverable errors.

These hooks enable developers to centralize error reporting and logging logic at the root of the application.

Listing A.12 illustrates the implementation of global error handling using hooks with `createRoot`, as defined in `mainWithErrorHooks.tsx`. To try it out, replace the content of `main.tsx` with the contents of `mainWithErrorHooks.tsx`. This modification to `main.tsx` demonstrates how to leverage `onCaughtError`, `onUncaughtError`, and `onRecoverableError` to globally manage errors.

**Listing A.12 Centralized error processing implementation with createRoot in mainWithErrorHooks.tsx**

```
function onCaughtError(error: any, errorInfo: ErrorInfo) {
  console.error('In onCaughtError', error, errorInfo.componentSta
}
function onUncaughtError(error: any, errorInfo: ErrorInfo) {
  console.error('In onUncaughtError', error, errorInfo.componentS
}
function onRecoverableError(error: any, errorInfo: ErrorInfo) {
```